

Reviewer Pack — Algebraic Connectivity Exponentiates DPLL Tree Size for Tsei...

Entry 2fa20a1ca794 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 09:46:42 UTC

Algebraic Connectivity Exponentiates DPLL Tree Size for Tseitin Formulas

Entry ID: 2fa20a1ca794 Verdict: INCONCLUSIVE Recorded: 2026-05-09 09:46:42 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): DPLL Search Tree

Statement:

For a Tse, the DPLL search tree size is at least $2^{\{c * \mu(G)\}}$ where $\mu(G)$ is the algebraic connectivity of G and $c > 0$ is a constant.

Rationale (proposer’s reasoning):

Algebraic connectivity quantifies graph expansion, which is known to correlate with resolution complexity. Higher expansion implies longer resolution proofs, as seen in expander graphs requiring exponential refutations. Thus, the DPLL tree size should reflect this exponential dependence.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3a29066e737b0128

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for k in range(i+1, n):
                if abs(A[k][i]) > abs(A[max_row][i]):
                    max_row = k
            A[i], A[max_row] = A[max_row], A[i]
            factor = Fraction(A[i][i], 1)
            for j in range(n):
                A[i][j] /= factor
            for k in range(n):
                if k != i:
                    factor = Fraction(A[k][i], 1)
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def determinant(A):
        n = len(A)
        det = Fraction(1, 1)
        for i in range(n):
            det *= A[i][i]
        return det

    def algebraic_connectivity(G):
        n = len(G)
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if G[i][j]:
                    L[i][j] = -1
                    L[j][i] = -1
```

```

L[i][i] = sum(G[i]) - 1

L = gaussian_elimination(L)
det_L = determinant(L)

return abs(det_L) / (2 * n**2)

def generate_tseitin_formula(n):
    literals = [f'x{i+1}' for i in range(n)]
    clauses = []
    for i in range(n):
        clauses.append([literals[i]])
        for j in range(i+1, n):
            clauses.append([-literals[i], -literals[j]])
            clauses.append([literals[i], literals[j]])
    return clauses

def dpll(clauses, assignment, model=[]):
    if not clauses:
        return True
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        new_assignment = assignment + [literal] if literal > 0 else assignment + [-literal]
        new_model = model + [(abs(literal), literal)]
        return dpll(clauses, new_assignment, new_model)

    literal = next((l for l in literals if l not in assignment and -l not in assignment), None)
    if not literal:
        return False

    new_assignment1 = assignment + [literal]
    new_assignment2 = assignment + [-literal]
    return dpll(clauses, new_assignment1) or dpll(clauses, new_assignment2)

n = random.randint(5, 40)
G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
for i in range(n):
    G[i][i] = 0

μ_G = algebraic_connectivity(G)
if μ_G == 0:
    return {
        "metric_name": "DPLL Tree Size",
        "metric_value": None,
        "instances_tested": n * (n - 1) // 2,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

clauses = generate_tseitin_formula(n)
dpll_tree_size = 0
for _ in range(30):
    if dpll(clauses, []):
        dpll_tree_size += 1

```

```

    return {
        "metric_name": "DPLL Tree Size",
        "metric_value": dpll_tree_size,
        "instances_tested": n * (n - 1) // 2,
        "conjecture_holds": dpll_tree_size >= 2**(0.5 *  $\mu_G$ ),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [random.randint(2, 10**9) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d23d2ec.py", line 124, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d23d2ec.py", line 104, in run_trial
    clauses = generate_tseitin_formula(n)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d23d2ec.py", line 67, in generate_tseitin_formula
    clauses.append([-literals[i], -literals[j]])
                  ~~~~~
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError` in `generate_tseitin_formula`, preventing data collection
| next: Fix the string-to-integer conversion error in `generate_tseitin_formula` and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	83015
2	propose	ollama_remote	qwen3:8b	0	0	91388
3	preregistration	ollama_remote	qwen3:8b	0	0	24237
4	novelty	ollama_remote	qwen3:8b	0	0	20761
5	novelty	ollama_remote	qwen3:8b	0	0	14477
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42680
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14250
8	judge	ollama_remote	qwen3:8b	0	0	17564

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 308372 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/2fa20a1ca794.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2fa20a1ca794.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2fa20a1ca794.tar.gz` (if generated)